

# Networking Fundamentals for DevOps

A practical guide to understanding how users reach an application—from a domain name in the browser to a Spring Boot service running on a server.

## The Big Picture

Suppose a Spring Boot application is deployed on AWS with the following configuration:

```
Server IP: 13.235.80.20
Application port: 8080
```

Technically, the application could be accessed through:

```
http://13.235.80.20:8080
```

However, users expect a simple and secure address such as:

```
https://coderarmy.in
```

The complete journey looks like this:

```
coderarmy.in
  ↓
DNS resolves the domain to an IP address
  ↓
13.235.80.20
  ↓
Nginx receives traffic on port 80 or 443
  ↓
Nginx forwards the request to port 8080
```

↓  
Spring Boot application

To understand this flow, we need to understand IP addresses, ports, protocols, DNS, load balancers, and reverse proxies.

# 1. IP Addresses and Ports

## IP Address

When two computers communicate over a network, they need a way to identify each other. An **IP address** is the network address of a machine.

Example:

142.250.183.14

Without a destination address, a computer would not know where to send its packets.

## Why an IP Address Is Not Enough

A single server can run several applications and services at the same time:

| Service                 | Common port |
|-------------------------|-------------|
| SSH                     | 22          |
| Nginx using HTTP        | 80          |
| Nginx using HTTPS       | 443         |
| Spring Boot application | 8080        |
| MySQL                   | 3306        |

The IP address identifies the machine, while the port identifies the service running on that machine.

IP address + Port

For example:

10.0.1.15:8080

This means: connect to the machine at `10.0.1.15` and communicate with the application listening on port `8080`.

## Important Ports for DevOps

You do not need to memorize hundreds of ports. These are the ones you will encounter frequently:

| Port              | Common use              |
|-------------------|-------------------------|
| <code>22</code>   | SSH                     |
| <code>53</code>   | DNS                     |
| <code>80</code>   | HTTP                    |
| <code>443</code>  | HTTPS                   |
| <code>3306</code> | MySQL                   |
| <code>5432</code> | PostgreSQL              |
| <code>6379</code> | Redis                   |
| <code>8080</code> | Common application port |

These are conventions, not strict limitations. An application can use another available port when configured to do so.

## What Does “Listening on a Port” Mean?

When Spring Boot starts and displays:

Tomcat started on port 8080

it means the operating system is allowing that application to receive traffic sent to port `8080`.

```
Incoming request
  ↓
Server IP address
  ↓
Operating system
  ↓
Port 8080
  ↓
Spring Boot application
```

On Linux, listening ports can be inspected with:

```
ss -tulpn
```

An older alternative is:

```
netstat -tulpn
```

`ss` is generally preferred on modern Linux systems.

## 2. Protocols

An IP address tells us **which machine** to contact, and a port tells us **which application** to contact. We still need rules that define how the two sides will communicate.

Those rules are called a **protocol**.

Common examples include:

- HTTP
- HTTPS
- SSH
- DNS

- TCP
- UDP

These protocols work at different networking layers. For practical DevOps work, the immediate focus is on what each protocol is used for and how it affects application communication.

---

## 3. TCP vs UDP

The basic question is:

Is reliable, ordered delivery more important, or is lower communication overhead more important?

### TCP

TCP provides **connection-oriented, reliable, and ordered communication**.

Before sending application data, the client and server establish a connection. TCP then handles concerns such as:

- Did the data arrive?
- Did it arrive in the correct order?
- Was any data lost?
- Should missing data be retransmitted?

TCP is commonly used where missing or incorrectly ordered data would cause a problem.

Examples include:

- SSH
- HTTP/1.1
- HTTP/2
- Database connections

If an application sends an instruction such as:

Transfer ₹10,000 to account XYZ

the application needs reliable delivery. Silently losing part of the communication is not acceptable.

## UDP

UDP does not establish the same type of connection or provide TCP's built-in delivery guarantees. The sender sends a datagram without waiting for the same reliability checks.

This produces less protocol overhead, but delivery, ordering, and retransmission are not guaranteed by UDP itself.

Common use cases include:

- DNS queries
- Voice over IP
- Online gaming
- Some streaming scenarios

## TCP and UDP Compared

| Feature            | TCP                              | UDP                               |
|--------------------|----------------------------------|-----------------------------------|
| Connection         | Connection-oriented              | Connectionless                    |
| Delivery guarantee | Yes                              | No built-in guarantee             |
| Ordered delivery   | Yes                              | No built-in guarantee             |
| Retransmission     | Yes                              | No built-in retransmission        |
| Overhead           | Higher                           | Lower                             |
| Common examples    | SSH, HTTP/1.1, HTTP/2, databases | DNS, VoIP, gaming, some streaming |

## 4. Public and Private IP Addresses

Cloud infrastructure commonly separates systems that must be reachable from the Internet from systems that should remain internal.

For example, a public web server may need Internet access, but a database normally should not be directly accessible to everyone.

## Public IP Address

A public IP address is globally routable over the Internet.

```
Internet
  ↓
13.235.80.20
```

Machines on the Internet can potentially reach it when network routing and firewall rules allow the traffic.

A public IP does not automatically make every service accessible. Security groups, firewalls, routing rules, and listening ports still control what can be reached.

## Private IP Address

Private IP addresses are used inside private networks and are not directly routable over the public Internet.

The common private IPv4 ranges are:

```
10.0.0.0      – 10.255.255.255
172.16.0.0   – 172.31.255.255
192.168.0.0  – 192.168.255.255
```

Addresses beginning with `10.x.x.x` are especially common in cloud infrastructure.

# 5. DNS Fundamentals

Humans prefer names such as:

```
google.com  
github.com  
coderarmy.in
```

Networks ultimately need IP addresses. The **Domain Name System**, or **DNS**, connects these two worlds by mapping domain names to network information.

```
coderarmy.in  
  ↓ DNS  
13.235.80.20
```

DNS is often called the **phonebook of the Internet**. A person's name leads to a phone number; similarly, a domain name leads to an IP address or another DNS value.

## 6. How DNS Resolution Works

Suppose a user enters:

```
www.example.com
```

The browser needs the server's IP address before it can connect. DNS resolution usually follows a chain rather than asking one magical server that knows every answer.

### Step 1: Browser and Operating-System Cache

The browser or operating system first checks whether it already has a valid cached answer.

If the domain was resolved recently and the cached record has not expired, the system can reuse it without performing the complete lookup again.

### Step 2: Recursive DNS Resolver



If the answer is not available locally, the machine asks a **recursive resolver**. The resolver may be operated by:

- An Internet service provider
- A company
- A cloud provider
- A public DNS service

Common public resolver addresses include:

```
8.8.8.8  
1.1.1.1
```

The resolver finds the answer on behalf of the client. It also checks its own cache before continuing through the DNS hierarchy.

## Step 3: Root Name Server

If the resolver does not have the answer cached, it starts at the DNS root.

DNS names can be understood from right to left:

```
.  
└─ com  
    └─ example.com  
        └─ www.example.com
```

The resolver asks the root server where it can find information about `.com`. The root server does not normally return the final website IP. It directs the resolver to the appropriate `.com` top-level domain servers.

## Step 4: TLD Name Server

**TLD** stands for **top-level domain**.

Examples include:

- `.com`

- `.org`
- `.net`
- `.in`
- `.io`

The resolver asks the `.com` TLD server which name servers are responsible for `example.com`. The TLD server points it toward the domain's authoritative name servers.

## Step 5: Authoritative DNS Server

The authoritative server stores the DNS records configured for the domain.

When asked for `www.example.com`, it can return an answer such as:

```
93.184.216.34
```

The recursive resolver returns this result to the client and usually caches it for future requests.

## Complete DNS Resolution Chain

```
Browser or operating system
    ↓
Recursive DNS resolver
    ↓
Root name server
    ↓
TLD name server (.com)
    ↓
Authoritative name server (example.com)
    ↓
IP address returned to the resolver
    ↓
IP address returned to the browser
```

Once the browser has the IP address, it can begin the actual network connection to the server.

## 7. DNS Caching and TTL

Repeating the complete root → TLD → authoritative lookup for every request would be inefficient. DNS results are therefore cached at several levels.

### TTL: Time to Live

The **TTL** tells DNS caches approximately how long they may reuse a record before requesting a fresh value.

Example:

```
example.com → 13.235.80.20  
TTL = 3600 seconds
```

A TTL of **3600** allows the record to be cached for approximately one hour.

### Why TTL Matters in DevOps

Suppose an application is moved from an old server to a new server:

```
Old IP: 13.235.80.20  
New IP: 3.110.55.10
```

Even after updating the DNS record, some users may temporarily reach the old server because their resolver still has the previous value cached.

This is one reason DNS changes do not always appear everywhere immediately.

### Low TTL vs High TTL

| TTL                                | Advantage                            | Trade-off                         |
|------------------------------------|--------------------------------------|-----------------------------------|
| Low, such as <b>60</b> seconds     | DNS changes can be observed sooner   | More frequent DNS queries         |
| High, such as <b>86400</b> seconds | Better caching and fewer DNS queries | Changes can take longer to appear |

Before a planned infrastructure migration, teams sometimes lower the TTL in advance. Once old cached entries expire, the DNS record can be changed with a shorter transition period.

---

## 8. Essential DNS Record Types

DNS stores different record types for different purposes. The most useful ones for DevOps are `A`, `AAAA`, `CNAME`, `MX`, `TXT`, and `NS`.

### A Record

An `A` record maps a hostname to an **IPv4 address**.

```
coderarmy.in
  ↓ A
13.235.80.20
```

A typical DNS configuration may look like:

```
Type:  A
Name:  @
Value: 13.235.80.20
```

In many DNS dashboards, `@` represents the root or apex domain—in this case, `coderarmy.in`.

An `A` record can also be created for a subdomain:

```
Type:  A
Name:  api
Value: 13.235.80.30
```

This maps:

```
api.coderarmy.in → 13.235.80.30
```

## AAAA Record

An **AAAA** record maps a hostname to an **IPv6 address**.

```
example.com
  ↓ AAAA
2001:db8:....
```

## CNAME Record

A **CNAME** record makes one hostname an alias of another hostname.

```
www.coderarmy.in
  ↓ CNAME
coderarmy.in
```

This avoids configuring the same IP address separately for both names.

Another example is:

```
blog.coderarmy.in
  ↓ CNAME
some-hosting-provider.example
```

The important difference is:

```
A record:      hostname → IP address
CNAME record:  hostname → another hostname
```

**www.coderarmy.in** and **coderarmy.in** are separate hostnames. Configuring the root domain does not automatically configure **www**. If **www** should lead to the same website, it needs an appropriate DNS record, commonly a **CNAME**.

## MX Record

**MX** stands for **Mail Exchange**. It tells mail systems where email for a domain should be delivered.

For an address such as:

```
hello@coderarmy.in
```

the domain's MX records may point to the servers operated by its email provider.

```
coderarmy.in
  ↓ MX
Mail provider
```

The application and email infrastructure can use different records under the same domain:

```
A record → application infrastructure
MX record → mail infrastructure
```

## TXT Record

A **TXT** record stores text data in DNS. It is frequently used for verification and email-security configuration.

Common use cases include:

- Google Workspace verification
- Microsoft service verification
- Domain ownership verification
- SPF configuration
- DKIM-related configuration
- Some Let's Encrypt validation methods

Example:

```
Type:  TXT
Name:  @
```

```
Value: google-site-verification=xyz123...
```

When a service finds the expected value in DNS, it can verify that the person completing the setup controls the domain's DNS configuration.

## NS Record

**NS** stands for **Name Server**. NS records identify the authoritative DNS servers for a domain.

```
coderarmy.in
  ↓ NS
ns1.dns-provider.com
ns2.dns-provider.com
```

Those authoritative name servers store the domain's other records, such as:

- **A**
- **AAAA**
- **CNAME**
- **MX**
- **TXT**

## DNS Record Summary

| Record       | Maps or stores                     | Common use                            |
|--------------|------------------------------------|---------------------------------------|
| <b>A</b>     | Hostname → IPv4 address            | Point a domain to an IPv4 server      |
| <b>AAAA</b>  | Hostname → IPv6 address            | Point a domain to an IPv6 server      |
| <b>CNAME</b> | Hostname → another hostname        | Create an alias such as <b>www</b>    |
| <b>MX</b>    | Domain → mail server               | Route email for the domain            |
| <b>TXT</b>   | Domain → text value                | Verification, SPF, DKIM-related setup |
| <b>NS</b>    | Domain → authoritative name server | Identify where the DNS zone is hosted |

## 9. Domain Registrar vs DNS Provider

A **domain registrar** is the company through which a domain is registered. A **DNS provider** hosts and serves the DNS records for that domain.

These do not have to be the same company.

```
Domain registrar
  ↓ NS records
DNS provider
  ↓ A / AAAA / CNAME / MX / TXT records
Application and mail infrastructure
```

The NS configuration connects the registered domain to the DNS provider that holds its authoritative zone.

## 10. Connecting a Domain to an Application

Assume the following deployment:

```
Platform: AWS EC2
Public IP: 13.235.80.20
Spring Boot port: 8080
Domain: coderarmy.in
```

### Step 1: Create the DNS Record

Configure an **A** record:

```
Type: A
Name: @
Value: 13.235.80.20
```

DNS can now resolve:

```
coderarmy.in → 13.235.80.20
```



## Step 2: The User Enters the Domain

When the user visits `coderarmy.in`, DNS returns `13.235.80.20`. The browser now knows which machine to contact.

## Step 3: Handle the Port Difference

The Spring Boot application is listening on `8080`, but URLs that omit an explicit port use the protocol's default port:

```
HTTP → port 80
HTTPS → port 443
```

Therefore:

```
http://coderarmy.in
```

normally sends traffic to:

```
13.235.80.20:80
```

It does not automatically connect to `13.235.80.20:8080`.

A reverse proxy such as Nginx solves this problem:

```
Internet
  ↓ port 80 or 443
Nginx
  ↓ port 8080
Spring Boot application
```

# 11. Subdomains

A domain can have multiple subdomains that point to different applications or infrastructure.

```
coderarmy.in      → Frontend
api.coderarmy.in  → Backend API
admin.coderarmy.in → Admin application
docs.coderarmy.in → Documentation
```

Example DNS configuration:

```
coderarmy.in
A
13.235.80.20

api.coderarmy.in
A
13.235.80.30
```

This allows related systems to be organized under a single domain while keeping their infrastructure independent.

## 12. Load Balancing Fundamentals

A single server has limited CPU, memory, network capacity, and available connections. As traffic grows, that server can become overloaded.

One solution is to run multiple copies of the application:

```
Server 1 → 10.0.1.10
Server 2 → 10.0.1.11
Server 3 → 10.0.1.12
```

This creates a new question: which server should receive each request?

A **load balancer** answers that question. It receives client traffic and distributes requests across multiple backend servers.

```
Users
  ↓
Load balancer
  └─ Server 1: 10.0.1.10
```

```
└─ Server 2: 10.0.1.11
└─ Server 3: 10.0.1.12
```

DNS points the application's domain to the load balancer:

```
api.coderarmy.in
  ↓ DNS
Load balancer
  ↓
Healthy backend server
```

The user does not need to know how many backend servers exist or which server processes a particular request.

## Why Use a Load Balancer?

### 1. Scalability

A system can be scaled in two basic ways:

```
Vertical scaling:
Increase the capacity of one server
2 CPUs → 8 CPUs → 32 CPUs

Horizontal scaling:
Increase the number of servers
1 server → 3 servers → 10 servers
```

Load balancers make horizontal scaling practical by distributing traffic among the available servers.

### 2. Availability

If an application depends on only one server, that server becomes a single point of failure.

With several backends, the load balancer can stop routing traffic to a failed instance and continue using the healthy ones.

```
Load balancer
├─ Server 1 ✓
├─ Server 2 ✗
└─ Server 3 ✓
```

### 3. Traffic Distribution

A load balancer uses a routing algorithm to avoid sending most requests to one server while leaving the others underused.

## 13. Load-Balancing Algorithms

### Round Robin

Round robin sends requests to each server in sequence.

```
Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A
Request 5 → Server B
Request 6 → Server C
```

It works well when backend servers have similar capacity and requests require roughly comparable work.

### Least Connections

Round robin does not consider how busy each server already is.

Suppose the current connection counts are:

```
Server A → 100 active connections
Server B → 20 active connections
Server C → 5 active connections
```

The least-connections algorithm sends the next request to Server C because it currently has the fewest active connections.

This can be useful when some requests or connections remain active much longer than others.

## Sticky Sessions

Suppose a user logs in through Server A, and Server A stores that user's session only in its local memory:

```
Session ID: abc123
User: Rohit
Role: ADMIN
```

If the next request is sent to Server B, Server B may not recognize the session.

With **sticky sessions**, the load balancer attempts to keep the same client connected to the same backend:

```
User A → Server 1 → Server 1 → Server 1
User B → Server 2 → Server 2 → Server 2
```

Sticky sessions can solve the immediate problem, but they also tie a user to one server. Modern systems often prefer stateless application instances and shared state.

For example:

```
Application servers
    ↓
Shared Redis session store
```

JWT-based authentication is another common approach. When the servers do not depend on local session state, requests can move freely between healthy instances.

## 14. Health Checks

A load balancer must know which backend servers are able to handle traffic. It does this by performing regular **health checks**.

For example:

```
GET /health
```

Possible results:

```
Server A → 200 OK → Healthy  
Server B → Timeout → Unhealthy  
Server C → 200 OK → Healthy
```

The load balancer sends traffic only to healthy backends. If Server B later recovers and starts passing its checks, the load balancer can automatically add it back into rotation.

Health checks are a core part of building highly available systems.

## 15. Layer 4 vs Layer 7 Load Balancing

### Layer 4 Load Balancing

Layer 4 load balancing works mainly with transport-level information such as:

- IP address
- Port
- TCP or UDP

It does not need to understand the full HTTP request.

```
Traffic for IP:443  
↓  
Selected backend server
```

It is relatively low-level and efficient.

## Layer 7 Load Balancing

Layer 7 load balancing understands application-level protocols such as HTTP. It can route requests using information such as:

- Hostname
- URL path
- HTTP method
- Headers
- Cookies

Example:

```
/api/products/* → Product Service  
/api/orders/*   → Order Service
```

This makes Layer 7 routing especially useful for web applications and microservice architectures.

## AWS Terminology

In AWS, two common services are:

| AWS service                     | Primary orientation |
|---------------------------------|---------------------|
| Application Load Balancer (ALB) | Layer 7             |
| Network Load Balancer (NLB)     | Layer 4             |

# 16. Reverse Proxy Fundamentals

A production application is commonly placed behind a **reverse proxy** such as Nginx.

Suppose Spring Boot runs internally on:

```
localhost:8080
```

Users access:

```
https://coderarmy.in
```

HTTPS traffic reaches port `443`. Nginx receives that public request and forwards it to the Spring Boot application on port `8080`.

```
User
  ↓ HTTPS on port 443
Nginx
  ↓ HTTP on port 8080
Spring Boot
```

The user communicates with the reverse proxy and does not need to know the application's internal address or port.

A reverse proxy and a load balancer are not identical concepts, but their capabilities can overlap. A reverse proxy sits in front of backend applications and forwards client requests. When it distributes those requests across multiple backends, it is also performing load balancing.

## Final Request Flow

The complete journey can now be understood as follows:

1. The user enters `https://coderarmy.in`.
2. DNS resolves `coderarmy.in` to a public IP address.
3. The browser connects to that IP on HTTPS port 443.
4. A reverse proxy or load balancer receives the request.
5. It checks the available backend servers and selects a healthy one.
6. It forwards the request to the Spring Boot application on its internal port.



7. The application processes the request and returns the response through the same path.

## Quick Revision

| Concept       | Purpose                                                            |
|---------------|--------------------------------------------------------------------|
| IP address    | Identifies a machine on a network                                  |
| Port          | Identifies a service on that machine                               |
| Protocol      | Defines the communication rules                                    |
| TCP           | Reliable and ordered transport                                     |
| UDP           | Low-overhead transport without built-in delivery guarantees        |
| Public IP     | Globally routable Internet address                                 |
| Private IP    | Address used inside a private network                              |
| DNS           | Converts domain names into network information                     |
| TTL           | Controls how long DNS records may be cached                        |
| Load balancer | Distributes traffic across backend servers                         |
| Health check  | Determines whether a backend should receive traffic                |
| Reverse proxy | Receives public requests and forwards them to backend applications |